COMP3702 Artificial Intelligence
Semester 2, 2026
Tutorial 2 - Sample Solutions

Exercise 2.1

Start to tackle this problem by working through the agent design components.

Problem description:
The 8-puzzle problem contains 9 cells stored in a grid. This can be represented as a string of 9 characters, with an '_' where the empty space is:

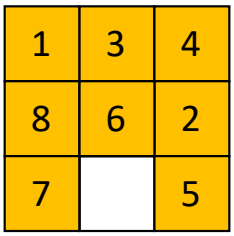

Figure 1: Example 8-puzzle state, which can be represented as the string 1348627_5

Action Space:
The action space only consists of 4 actions that are, move the blank space: {`up`, `down`, `left`, `right`}. Note that some actions are invalid, e.g. in the example above, moving the blank space `down` would be invalid, as there isn't anything there.

One way to identify invalid actions programmatically is by observing the row and column of the blank space. Using 1-indexing, the first element in the string will be at index 1.

- If the blank space is in the top row, then its index in the string is less than 4. In this case, an `up` action would be invalid. Similarly, if its index is greater than 6, then it is in the bottom row, and a `down` action would be invalid.
- `left` and `right` invalid moves can be determined by the modulus operator. If the blank index % 3 is 0, then it is in the right column, and a `right` action is invalid. If the index % 3 is equal to one, then it is in the left column, and the `left` action is invalid.

BFS and DFS:
There are a total of 9! states in the 8-puzzle problem (= 362,880). Note since only the order matters, there are actually only 8! = 40,320 states, however this is still very large! It would take a long time to store each one of them. Instead, they can be created as the search is performed.

A node in a tree can be represented as in Figure 2.

In BFS, the successors of a node get added to the frontier which is a First In First Out (FIFO) queue. This can be implemented in Python using `container.append(next_node)`,

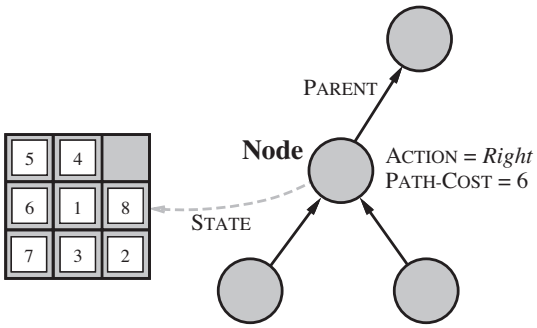

Figure 2: Example of a node which represents its STATE, stores its PARENT, the ACTION that was performed from the parent to arrive at the node, and the PATH-COST to arrive at that node (Source: Russell & Norvig textbook)

and `container.pop(0)`. The successors will be the neighbours of the current state. For each state there can be a maximum of 4 neighbours.

e.g. 1348627_5 has 3 neighbours: (1348_2765, 134862_75, 13486275_)

1348627_5 would be the first state to get constructed and added to the queue. It is checked whether it is the goal node, gets marked as expanded, and then its neighbours get constructed and added to the queue.

The pseudocode for BFS is shown in Figure 3. Note that this uses the visited/reached set (i.e. tracking nodes that have been generated, but may not yet have been expanded/explored yet) for tracking states which is more efficient than using an explored/expanded set.

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
  node ← NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node
  frontier ← a FIFO queue, with node as an element
  reached ← {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if problem.IS-GOAL(s) then return child
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure
```

Figure 3: Pseudocode for Breadth-First-Search on a graph (Source: Russell & Norvig textbook)

Alternatively, see P&M's discussion of BFS: `https://artint.info/3e/html/ArtInt3e.Ch3.S5.html`.

An example implementation is available on Blackboard.

In DFS, the same procedure is run, but the successors of the node get added to the frontier which operates as a Last In First Out (LIFO) stack. A LIFO frontier means that the most recently generated nodes are expanded first. This must be the deepest unexpanded node because it is one deeper than its parent – which, in turn, was the deepest unexpanded node when it

was selected. This can be implemented in Python using `container.append(next_node)`, and `container.pop()`. To avoid cycles, you can either check for cycles within the path of a generated node, or keep track of nodes already visited plus their path-cost as a dictionary where the key:value = state: path_cost, and only revisit state if the path-cost is lower. An alternative implementation is to implement depth-first search with a recursive function that calls itself on each of its children in turn.

Exercise 2.2

Run your puzzle solver to generate the Sequence of Actions, Number of Steps, and Time taken.

Example values (with goal-check on generation):

- startState = 1348627_5; goalState = 1238_4765
  BFS: steps = 5, visited = 22, time = 0.006s;
  DFS: steps = 836, visited = 854, time = 0.0172s
  Implementation 2:
  BFS: time = 0.00016673564910888673 seconds, #actions = 5, actions = [2, 1, 2, 0, 3]
  DFS: time = 0.18844389915466309 seconds, #actions = 51547
- startState = 281_43765; goalState = 1238_4765
  BFS: steps = 9, visited = 219; time = 0.03s;
  DFS: steps = 69434, visited = 72441, time = 253.44s
  Implementation 2:
  BFS: time = 0.001159815788269 seconds, #actions = 9, actions = [2, 1, 1, 3, 0, 0, 2, 1, 3]
  DFS: time = 0.3581709861755371 seconds, #actions = 59123
- startState = 281463_75; goalState = 1238_4765
  BFS: steps = 12, visited = 1120; time = 0.266s
  DFS: steps = 82141, visited = 86490; time = 361.0s
  Implementation 2:
  BFS: time = 0.003334898948669 seconds, #actions = 12, actions = [1, 2, 0, 2, 1, 1, 3, 0, 0, 2, 1, 3]
  DFS: time = 0.08257007598876953 seconds, #actions = 27962

The solution for all 3 cases is a relatively small distance away from the start node. BFS takes a very small number of steps, while DFS takes a very large number of steps. This can happen in cases where the solution is a small number of actions away, as BFS prioritises an optimal number of actions, while DFS is not guaranteed to be optimal, and expands its search to bigger depths and larger sequences.

BFS also requires significantly less time to solve the problem than DFS in each of the given cases. This result is in line with the time complexity of each algorithm – the time complexity of BFS has exponential dependence on d, the depth of the minimum depth goal, while the time complexity of DFS has exponential dependence on the m, the maximum depth of the search tree.

Recall time complexity for BFS is $O(b^d)$, and for DFS is $O(b^m)$. Time complexity can be calculated by performing regression on this data (e.g. try using a curve-fitting tool). It is possible that the results may not fit the expected complexity for BFS and DFS. This can be

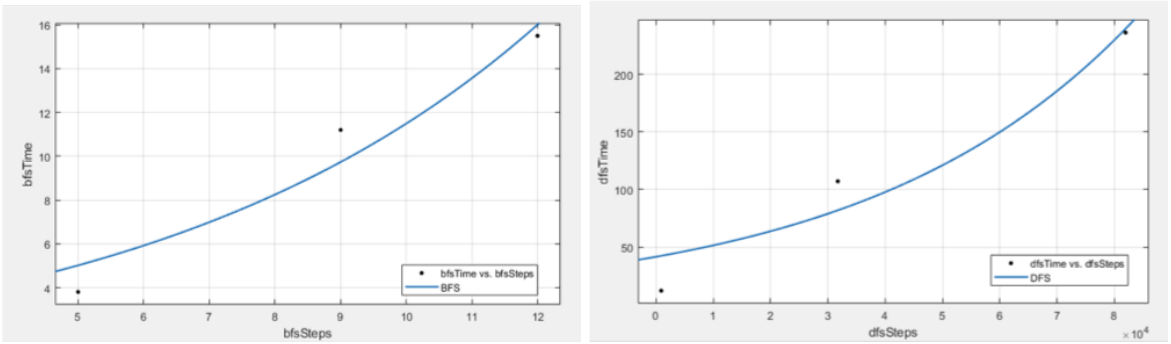

Figure 4: Time complexity for BFS vs DFS

the result of overhead in the calculations, or lack of datapoints – try adding more tests to the regression curve.

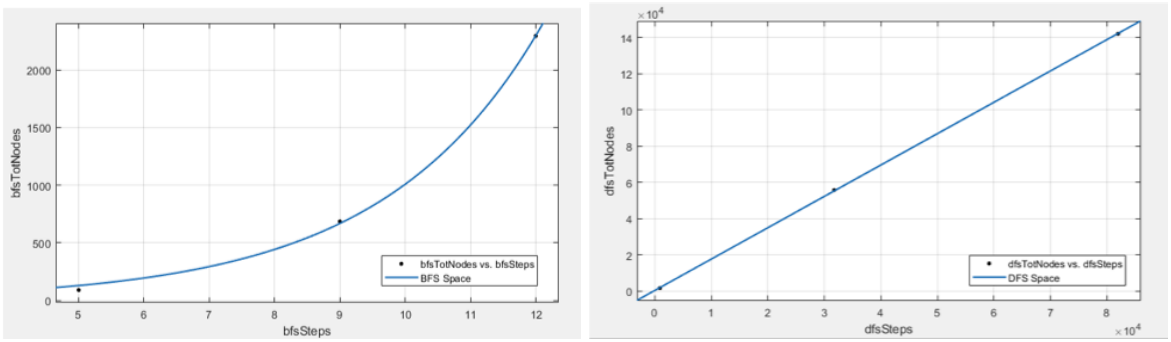

Figure 5: Space complexity for BFS vs DFS)

Space can be approximated by observing the maximum number of nodes in the queue for BFS or number of nodes in the stack for DFS at any time.

Space complexity for BFS is $O(b^d)$, the number of nodes expanded, and for DFS is $O(bm)$, for the 8-puzzle problem, the branching factor is 4. For DFS, the number of nodes in the stack is increased with the maximum depth of the tree – you can try changing the max depth in the code and observe the results. This is called depth-limited search. A combination of DFS and BFS can be implemented called Iterative-deepening depth first search (IDDFS), where we use depth-limited search with increasing search depth.

Exercise 2.3

(a) Recall that BFS performs a complete search, given a finite branching factor. Thus, BFS will search all possible combinations of paths within the search space, and eventually run out. When the search has run out of paths to search it can conclude that there is no solution, thus BFS can output the right answer.

DFS can also perform a complete search if m and b are finite and states are not revisited. Although the path may not be optimal, DFS can also output the right result.

(b) (See supporting notes on tutorial worksheet)

Parity can be used to determine if the goal node is reachable from the start node, if the start and goal have the same parity, it is reachable, otherwise it is not. This can significantly improve the size of the state space, as half of the states can be ignored immediately, as they are unreachable. The state space for the 8-puzzle problem is 9!, that's a big number. Using parity checking, this can be reduced by a factor of 2.

Parity can be calculated by finding the number of inversions of a state mod 2.

- The number of inversions of tile-i ($N(s, i)$) is the number of tiles that appear after tile-i that have a smaller value than tile-i.
- The parity is the total sum of the inversions of each tile, mod 2
  Parity $= \sum_{i=1}^{n} N(s, i) \bmod 2$

Exercise 2.4

Note, UCS can be implemented in python using a priority queue object `pq` (e.g. using `heapq`, or `queue.PriorityQueue`), where `pq.push(key, value)` inserts (key, value) into the queue, and `pq.pop()` returns the key with the lowest value and removes it from the queue.

a) Yes

b) They should return the same optimal path. However, in some implementations there might be a difference in when the goal test is applied. e.g. in BFS the goal test is usually applied to each node when it is generated rather than when it is selected for expansion. In Uniform-Cost-Search, the goal test is usually applied to a node when it is selected for expansion (taken off the frontier) rather than when it is first generated. The reason is that the first goal node that is generated may be on a suboptimal path, such as when there are different costs for different actions.